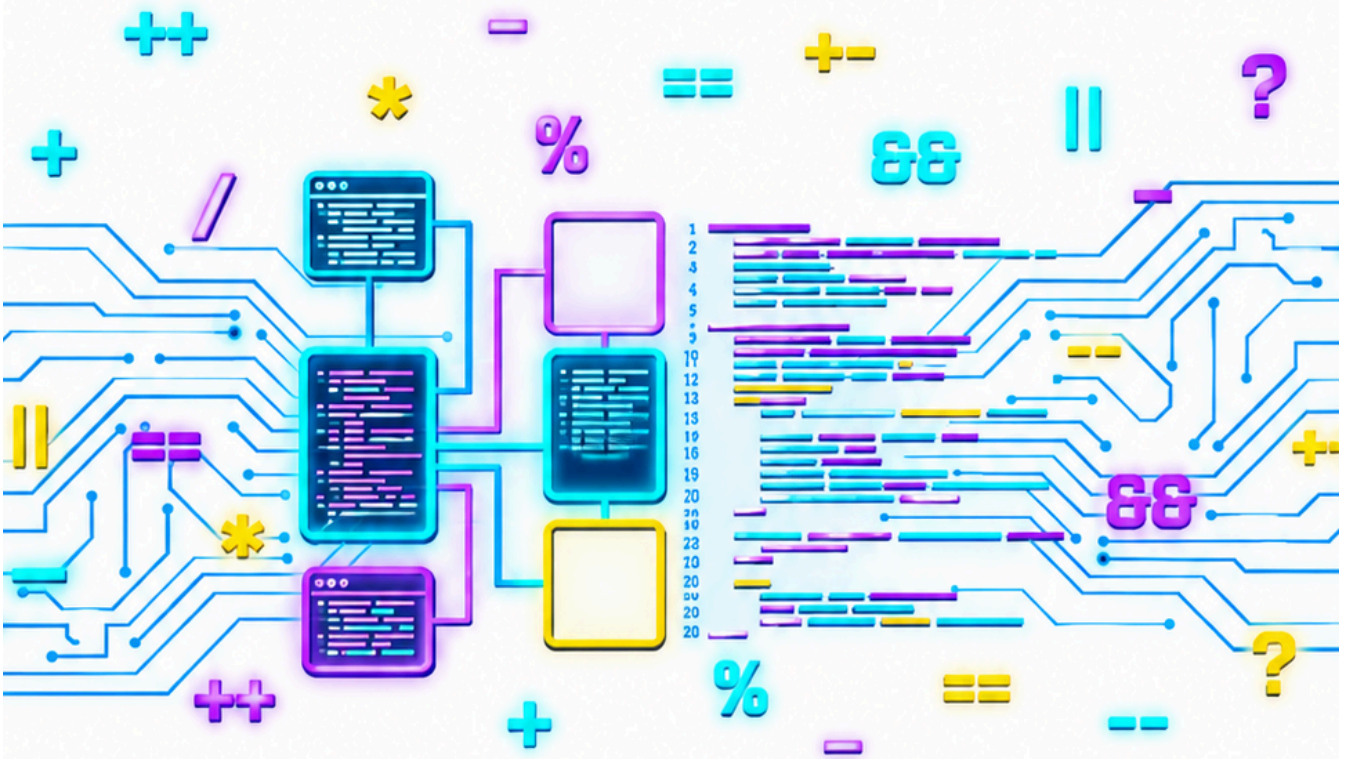
CHAPTER 4

OPERATORS AND EXPRESSIONS



JAVA PROGRAMMING SERIES

INDEX

SL. No.	Topic	Page No.
1	Java Operations	3
2	Arithmetic Operators	4
3	Relational and Logical Operators	5
4	Assignment and Shorthand Operators	7
5	Increment and Decrement Unary Operators	8
6	The Conditional (Ternary) Operator	9
7	Bitwise Operators	10
8	Special Operators	16
9	Unary Operators	17
10	Binary Operators	18
11	Dynamic Memory Allocation (new) Keyword	19
12	String Manipulation	20
13	Java Expressions	21
14	Mathematical Functions	26

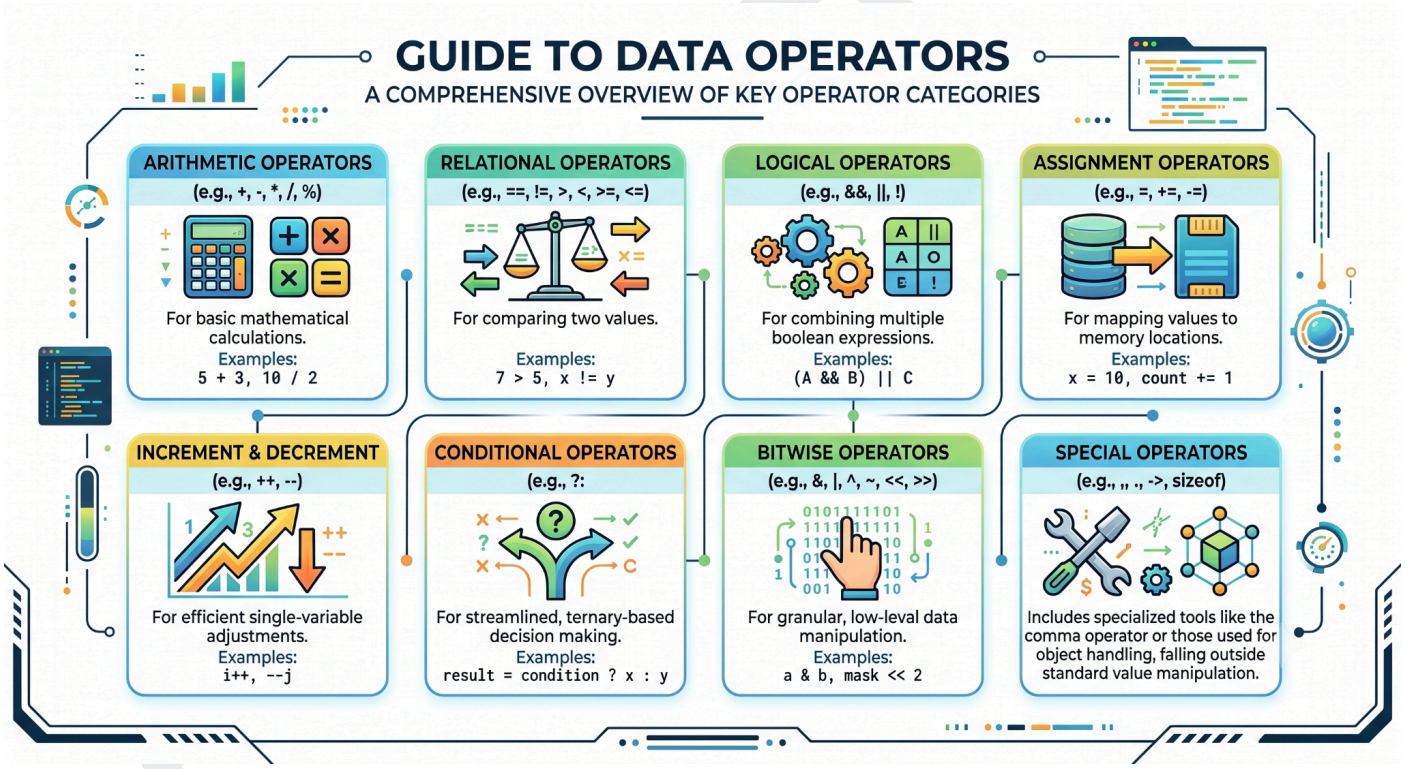
ABHILASH B J

OPERATORS AND EXPRESSIONS

Java Operations

In the Java programming environment, an operator is a symbolic representation of a specific instruction that tells the computer to perform mathematical or logical manipulations. Strategically, these operators serve as the fundamental bridge between raw data variables and functional logic, enabling the transformation of static values into dynamic computational results. Without operators, variables would remain inert; with them, developers can construct the complex logic required to drive modern software applications.

Java operators are categorized into eight primary taxonomies to manage different computational needs:



Arithmetic Operators

Arithmetic operators are the bedrock of most algorithmic development, facilitating the construction of mathematical expressions that drive data processing. In a technical architecture, these operators allow for the translation of business logic into executable numerical code.

Operator Symbol	Formal Name	Functional Description
+	Addition	Adds two operands together.
-	Subtraction	Subtracts the second operand from the first.
*****	Multiplication	Multiplies the two operands.
/	Division	Divides the first operand by the second.
%	Remainder	Returns the integer remainder after division.

Applied Logic & Result Analysis

```
// Usage of Arithmetic Operators
class Arithmetic
{
    public static void main (String agr[])
    {
        int x, y;
        x = 10;
        y = 3;
        System.out.println("Addition=" + (x + y));
        System.out.println("Subtraction=" + (x - y));
        System.out.println("Multiplication=" + (x * y));
        System.out.println("Division=" + (x / y));
        System.out.println("Remainder=" + (x % y));
    }
}
```

Program Output

```
javac Arithmetic.java
java Arithmetic

Addition=13
Subtraction=7
Multiplication=30
Division=3
Remainder=1
```

ABHILASH B J

In Java integer mathematics, the **Division (/)** operator performs what is known as **truncation**. This means the result is truncated toward zero, discarding any fractional part to return a whole number. Complementing this, the **Remainder (%)** operator captures the leftover value from that division. Here, since 3 goes into 10 three times with 1 left over, the result is 1. While these operators calculate specific values, relational operators are necessary to compare those values to drive program flow.

Relational and Logical Operators

The strategic importance of Boolean logic in software flow control cannot be overstated. Relational operators allow a system to compare two discrete values, while logical operators synthesize these comparisons into complex, compound decision-making frameworks.

Relational Comparison

Relational operators form expressions that evaluate to a boolean result: either `true` or `false`.

Operator	Description	Result (x=10, y=3)
>	Greater than	true
<	Less than	false
>=	Greater than or equal to	true
<=	Less than or equal to	false
==	Equal to	false
!=	Not equal to	true

```
// Usage of Relational Operators
class Relational
{
    public static void main (String agr[])
    {
        int x,y;
        x=10;
        y=3;
        System.out.println("(x>y)="+ (x>y));
        System.out.println("(x>=y)="+ (x>=y));
        System.out.println("(x<y)="+ (x<y));
        System.out.println("(x<=y)="+ (x<=y));
```

ABHILASH BJ

```
        System.out.println("(x==y)="+ (x==y));
        System.out.println("(x!=y)="+ (x!=y));
    }
}
```

Program Output:

```
javac Relational.java
java Relational

(x>y)=true
(x>=b)=true
(x<y)=false
(x<=y)=false
(x==y)=false
(x!=y)=true
```

Logical Synthesis

Logical operators—**Logical NOT (!)**, **Logical AND (&&)**, and **Logical OR (||)**—combine relational expressions into a single boolean outcome.

- **&& (Logical AND):** Returns true only if both expressions are true. Example: `(x == 10) && (y >= 3)` evaluates to **true**.
- **|| (Logical OR):** Returns true if at least one expression is true. Example: `(x == 5) || (y >= 4)` evaluates to **false**.
- **! (Logical NOT):** Reverses the boolean state. Example: `!(x <= 5)` evaluates to **true** because (10 <= 5) is false.

```
// Usage of Logical operators
class Logical
{
    public static void main(String arg[])
    {
        int x=10,y=3;
        System.out.println((x==10) && (y>=3));
        System.out.println((x==5) || (y>=4));
        System.out.println(!(x<=5));
    }
}
```

ABHILASH B J

Program Output:

```
javac Logical.java
java Logical

true
false
true
```

Assignment and Shorthand Operators

Variable management has evolved to favor code conciseness without sacrificing clarity. Assignment operators map values to memory addresses, and the evolution of shorthand notation allows developers to perform operations and assignments in a single, less verbose step.

The Assignment Mechanism

The standard assignment operator **'='** assigns the value of an expression to a variable. Java also provides shorthand (or compound) assignment operators to streamline updates to a variable's state.

Normal Assignment	Shorthand Assignment	Description
x = x + 5	x += 5	Addition and assignment
x = x - 5	x -= 5	Subtraction and assignment
x = x * 5	x *= 5	Multiplication and assignment
x = x / 5	x /= 5	Division and assignment
x = x % 5	x %= 5	Remainder and assignment
x = x << 2	x <<= 2	Left shift and assignment
x = x >> 2	x >>= 2	Right shift and assignment
x = x & 2	x &= 2	Bitwise AND and assignment
x = x | 2	x |= 2	Bitwise OR and assignment
x = x ^ 2	x ^= 2	Bitwise XOR and assignment

ABHILASH B J

Increment and Decrement Unary Operators

Unary operators are unique in that they operate on a single operand. The increment (++) and decrement (--) operators are strategically vital because of their "side effects"—they allow a variable to be modified and evaluated in the same step, though the timing of this change depends on the syntax used.

Functional Differentiation

- **Prefix (++x, --x):** The variable is modified *before* the expression is evaluated. The operation uses the new, updated value.
- **Postfix (x++, x--):** The expression is evaluated using the *original* value, and the modification occurs immediately afterward.

```
//Usage of prefix and postfix operators
public class PrePost
{
    public static void main(String[ ] args)
    {
        int x = 10;
        int y = 20;
        System.out.println(++x);
        System.out.println(x++);
        System.out.println(x);
        System.out.println(--y);
        System.out.println(y--);
        System.out.println(y);
    }
}
```

Program Output

```
javac PrePost.java
java PrePost

11
11
12
19
19
18
```

ABHILASH B J

Code Execution Breakdown

- **`++x` (Prefix increment):** Increments `x` from 10 to 11 first, then prints **11**.
- **`x++` (Postfix increment):** Prints the current value **11** first, then increments `x` to 12.
- **`x`:** Prints the updated value **12**.
- **`--y` (Prefix decrement):** Decrements `y` from 20 to 19 first, then prints **19**.
- **`y--` (Postfix decrement):** Prints the current value **19** first, then decrements `y` to 18.
- **`y`:** Prints the updated value **18**.

The Conditional (Ternary) Operator

The Conditional Operator is Java's only ternary operator, so named because it requires three operands. It serves as a high-efficiency alternative to standard if-else blocks for simple variable assignments.

Syntax and Evaluation Logic

The general form is: `variable = expression-1 ? expression-2 : expression-3;`

The evaluation follows a strict three-step logic:

1. **Evaluate expression-1:** A boolean condition is tested.
2. **Assign result if true:** If expression-1 is true, expression-2 is evaluated and its value is assigned.
3. **Assign result if false:** If expression-1 is false, expression-3 is evaluated and its value is assigned.

Real-World Application Analysis

Maximum of two variables where **x = 5** and **y = 10**, consider: `z = (x > y) ? x : y;`

In this case, the condition `(5 > 10)` evaluates to **false**. Following the evaluation logic, the operator skips the first result (`x`) and selects the second (`y`). Consequently, the value **10** is assigned to `z`. This provides a clean, one-line solution for logic that would otherwise require multiple lines of code.

ABHILASH B J

Bitwise Operators

Bitwise operators provide the highest level of control by allowing developers to manipulate data at its most granular level—the individual bit. This is strategically essential for high-performance computing, encryption, and direct hardware interaction.

These operators treat data not as integers or booleans, but as binary sequences. The Bitwise AND (&), OR (|), and XOR (^) operators compare bits of two operands, while the Bitwise NOT (~) operator inverts the bits of a single operand. Additionally, Java provides bit-shift operators, including Left Shift (<<), Right Shift (>>), and the Unsigned Right Shift (>>>), to move bit patterns across a memory word.

By mastering the full spectrum of operators from high-level conditional logic to low-level bitwise shifts a developer gains the ability to write Java code that is mathematically sound, logically robust, and highly optimized for performance.

Operator Inventory

Operator	Description
`&`	Bitwise AND
`|`	Bitwise OR
`^`	Bitwise Ex-OR
`~`	Bitwise complement
`<<`	Left shift
`>>`	Right shift
`>>>`	Zero fill right shift / Unsigned right shift
`<<=`	Left shift assignment
`>>=`	Right shift assignment
`>>>=`	Zero fill right shift assignment / Unsigned right shift assignment

ABHILASH BJ

The Logic Gate Baseline

The mathematical foundation of bit manipulation is rooted in universal truth tables. These tables define the resulting state of a bit based on the input values (x and y) and serve as the baseline for all bitwise logic in the Java Virtual Machine (JVM).

X	Y	~X	X & Y	X | Y	X ∧ Y
0	0	1	0	0	0
0	1	1	0	1	1
1	0	0	0	1	1
1	1	0	1	1	0

These fundamental logic gates translate mathematical theory into specific binary operations, where Java applies operand promotion to ensure calculations are performed at a minimum of int precision.

Logical Bitwise Operators: AND, OR, and XOR

Bitwise AND (&)

The Bitwise AND operator returns a 1 only if both operand bits are 1. Architecturally, this is utilized to "clear" or "mask" specific bits. By ANDing a value with a mask, you can isolate a subset of bits for inspection.

Calculation Example:

```
int x = 4, y = 5;
int z = x & y;
```

Variable	Binary Representation
Binary value of x (4)	`0000 0100`
Binary value of y (5)	`0000 0101`
Result z (4)	**`0000 0100`**

ABHILASH B J

Bitwise OR (|)

The Bitwise OR operator returns a 1 if either (or both) operand bits are 1. It is primarily used to "set" specific bits to 1, regardless of their current state, making it the standard choice for combining configuration flags.

Calculation Example:

```
int x = 4, y = 5;
int z = x | y;
```

Variable	Binary Representation
Binary value of x (4)	`0000 0100`
Binary value of y (5)	`0000 0101`
Result z (5)	**`0000 0101`**

Bitwise Ex-OR (^)

The Bitwise Exclusive OR (XOR) operator returns a 1 only if the operand bits are different. This unique parity-checking behavior makes it a critical tool for simple encryption, checksums, and toggling specific bit states (flipping a 1 to 0 or vice versa).

Calculation Example:

```
int x = 4, y = 5;
int z = x ^ y;
```

Variable	Binary Representation
Binary value of x (4)	`0000 0100`
Binary value of y (5)	`0000 0101`
Result z (1)	**`0000 0001`**

ABHILASH B J

Bitwise Complement and Shift Mechanics

Beyond logical gates, Java provides mechanics for bitwise inversion and positional shifting. These operations represent a bridge between logic and arithmetic, as shifting bits effectively performs rapid exponentiation-based calculations.

Bitwise Complement (~)

The Bitwise Complement is a unary operator that inverts every bit in its operand. Java utilizes the **2's Complement** system for signed integers. To determine the result of `~x`, the system follows this logic:

1. **Invert all bits:** 0 → 1 and 1 → 0.
2. **Interpretation:** The resulting pattern is interpreted as a negative value. To find the magnitude, invert the bits back and add 1 to the Least Significant Bit (LSB).

Example:

```
int x = 4;
int y = ~x;
```

- x (4) in binary: `0000 0100`
- ~x (Inversion): `1111 1011`
- **Result y:** -5.

Formula: `y = -(x + 1)`

Bit-Shifting Dynamics

Left Shift (<<)

Pushes bits to the left by n positions. The vacated bits on the right are filled with zeros.

- **Arithmetic Equivalence:** $x \times 2^n$.
- **Example:** `x = 4 << 2;` (Result: 16)
- **Binary:** `0000 0100` becomes `0001 0000`.

Right Shift (>>)

Pushes bits to the right by n positions. This is a **signed shift** (or arithmetic shift); it preserves the sign bit (the leftmost bit) to maintain the numerical sign of the value.

- **Arithmetic Equivalence:** $x / 2^n$.
- **Example:** `x = 4 >> 2;` (Result: 1)
- **Binary:** `0000 0100` becomes `0000 0001`.

ABHILASH D J

Zero Fill Right Shift / Unsigned Right Shift (`>>>`)

Exclusive to Java, this **unsigned shift** pushes bits to the right but always fills the vacated leftmost positions with zeros, regardless of the sign bit. This is essential when treating integers as pure bit patterns rather than signed numbers.

Compound Bitwise Assignment Operators

Compound assignment operators streamline code by performing a bitwise operation and updating the variable in a single, atomic-like step. This reduces verbosity in complex manipulation algorithms.

Note: *While some documentation suggests a syntax like* `variable = expression <<= n;`*, this is redundant. The correct Java syntax is a standalone assignment.*

Operator	Standalone Syntax	Operational Logic
Left Shift Assignment	`x <<= n;`	x = x <<l n
Right Shift Assignment	`x >>= n;`	x = x >> n
Zero Fill Right Shift Assignment	`x >>>= n;`	x = x >>> n

Operational Evidence: If `int x = 4;` and we execute `x >>= 2;`, the variable `x` is modified in memory to `1`. If we then execute `x <<= 4;`, `x` is updated to `16`.

Programmatic Implementation and Output Analysis

```
// Usage of Bitwise Operators
class Bitwise
{
    public static void main(String args[])
    {
        int x = 4;
        int y = 5;
        int z = 0;
        z = x & y;
        System.out.println("x & y = " + z); // 4
        z = x | y;
        System.out.println("x | y = " + z); // 5
        z = x ^ y;
```

ABHILASH BJ

```
        System.out.println("x ^ y = " + z); // 1
        z = ~x;
        System.out.println("~x = " + z); // -5
        z = x << 2;
        System.out.println("x << 2 = " + z); // 16
        z = x >> 2;
        System.out.println("x >> 2 = " + z); // 1
        z = x >>> 2;
        System.out.println("x >>> 2 = " + z); // 1
        x <<= 2; // Corrected Assignment
        System.out.println("x <<= 2 = " + x); // 16
        x >>= 2; // Corrected Assignment
        System.out.println("x >>= 2 = " + x); // 1
    }
}
```

Output and Analysis: The output confirms that `~4` results in `-5` due to 2's complement logic. Note the shifting: `x << 2` results in `16` because the bit at the 2^2 position (4) moved to the 2^4 position (16).

```
// Usage of Bitwise Operators for complex patterns
class Bitwise
{
    public static void main(String args[])
    {
        int a = 60;    /* 60 = 0011 1100 */
        int b = 13;    /* 13 = 0000 1101 */
        int c = 0;

        c = a & b;     /* 12 = 0000 1100 */
        System.out.println("a & b = " + c);

        c = a | b;     /* 61 = 0011 1101 */
        System.out.println("a | b = " + c);

        c = a ^ b;     /* 49 = 0011 0001 */
        System.out.println("a ^ b = " + c);

        c = ~a;        /* -61 = 1100 0011 */
        System.out.println("~a = " + c);

        c = a << 2;    /* 240 = 1111 0000 */
        System.out.println("a << 2 = " + c);
```

ABHILASH B J

```
        c = a >> 2;    /* 15 = 0000 1111 */
        System.out.println("a >> 2 = " + c);

        c = a >>> 2;   /* 15 = 0000 1111 */
        System.out.println("a >>> 2 = " + c);
    }
}
```

Special Operators

Java includes specific operators for structural navigation and object handling that ensure type-safety and object-oriented integrity.

- **Comma Operator (`,`):** Primarily used as a separator in variable declarations and `for` loop control expressions.
- **Dot Operator (`.`):** The member access operator, used to navigate object hierarchies and invoke methods or access fields.
- **`instanceof` Operator:** A critical tool for Runtime Type Identification (RTTI). It checks if an object is an instance of a specific class or interface, returning a boolean. This is the cornerstone of safe downcasting and polymorphic behavior.

Operator	Function	Application Example
Comma (,)	Separates expression lists within a single statement; often used in loops and assignments.	`for(i=1, j=15; j>10; i++, j--)`
Dot (.)	Accesses variables, methods, classes, or packages.	`student.mark` (Accessing the variable "mark")

Type Evaluation with instanceof

The `instanceof` operator provides a robust mechanism for object verification during the execution phase. It allows the system to determine if an object is an instance of a specific class or a subclass, which is essential for safe type casting. This operator performs a logical check and returns a boolean value: `true` if the object on the left-hand side matches the class type on the right-hand side, and `false` otherwise. For example, the expression `circle instanceof Shape` evaluates whether the object `circle` belongs to the `Shape` inheritance hierarchy.

ABHILASH B J

Unary Operators

Unary operators are essential for performing efficient state changes and logical inversions within a program. Because they act upon a single operand to produce a new value, they are often used for high-frequency operations like incrementing counters, toggling state, or negating numerical values.

Functional Taxonomy

The following table categorizes the common unary operators available in Java and their specific technical effects on operands:

Operator	Meaning	Technical Effect
+	Unary plus operator	Indicates a positive value (numbers are positive by default).
-	Unary minus operator	Negates an expression or value.
++	Increment operator	Increases the value of the operand by 1.
--	Decrement operator	Decreases the value of the operand by 1.
!	Logical complement	Inverts the value of a boolean operand.

Implementation Analysis

To understand how these operators alter variables in real-time, consider the following program implementation

```
// Usage of Unary Operators
class Unary
{
    public static void main(String[] args)
    {
        int x = 10;
        int y = 20;
        int c;

        c = +x;
        System.out.println("+x = " + c);
        c = -y;
        System.out.println("-y = " + c);
        c = ++x;
        System.out.println("++x = " + c);
        c = --y;
        System.out.println("--y = " + c);
    }
}
```

ABHILASH B J

Output Evaluation:

- **+x = 10**: The unary plus maintains the positive state of the variable.
- **-y = -20**: The unary minus negates the initial value of 20.
- **++x = 11**: The increment operator increases 10 to 11.
- **--y = 19**: Despite the output label in the source example reading `++y`, the decrement operation (`--y`) successfully reduces the value from 20 to 19.

While unary operators handle single-operand logic, the broader scope of data interaction is managed through Binary operators.

Binary Operators

Binary operators are the primary workhorses of Java computation, designed to handle interactions and perform calculations between two distinct operands. They form the basis of most algorithmic logic, allowing for both mathematical precision and logical comparisons.

Categorization of Binary Operations

Binary operations in Java are generally classified into five strategic categories:

1. **Arithmetic:** For mathematical calculations.
2. **Relational:** For comparing values (e.g., `x > y`).
3. **Logical:** For combining boolean expressions.
4. **Assignment:** For storing values in variables (e.g., `x = y`).
5. **Bitwise:** For manipulating data at the bit level.

Arithmetic Implementation

The following implementation demonstrates the practical application of binary arithmetic operators:

```
// Usage of Binary Operators
class Binary
{
    public static void main (String agr[])
    {
        int x, y;
        x = 10;
        y = 3;
        System.out.println("Addition = " + (x + y));
        System.out.println("Subtraction = " + (x - y));
        System.out.println("Multiplication = " + (x * y));
```

ABHILASH B J

```
        System.out.println("Division = " + (x / y));
        System.out.println("Remainder = " + (x % y));
    }
}
```

Arithmetic Results and Logic:

- **Addition (13):** Combines the two values.
- **Subtraction (7):** Finds the difference between operands.
- **Multiplication (30):** Scales the first operand by the second.
- **Division (3):** This performs **Integer Division**. In Java, when two integers are divided, the fractional part is truncated, not rounded. Thus, 10 divided by 3 equals 3.
- **Remainder (1):** Calculates the leftover value after division.

The "So What?" Factor: Arithmetic operators are more than simple math; they are foundational to logic. For instance, the **Remainder (%)** operator is a critical tool for developers to determine if a number is even or odd (e.g., `x % 2 == 0`) or to implement "wraparound" logic when navigating circular arrays or buffers.

ABHILASH B J

Dynamic Memory Allocation and the 'new' Keyword

In object-oriented programming, the ability to allocate memory at runtime is a strategic necessity. This allows programs to be flexible and efficient, creating objects only when they are required by the application's execution state.

Mechanism of the 'new' Keyword

The `new` operator is the primary tool for object creation. Its function is to dynamically allocate memory at runtime. When `new` is invoked, the system sets aside memory for a specific class instance and then calls a **Constructor**—a special method used to initialize the new object.

Syntax: `classname objectname = new classname();`

Example: `Fruits ob1 = new Fruits();`

In this instance, memory is explicitly allocated for a new object named `ob1`. The `Fruits()` call is the execution of the class **constructor**, which prepares the object for use. While `new` handles the creation of objects, specialized methods are required for manipulating specific object types like Strings.

String Manipulation

String handling is a critical aspect of Java development. Because Strings in Java are **immutable**—meaning they cannot be changed once created—any concatenation operation actually forms a completely new String object in memory.

Method Comparison

Java provides two primary architectural approaches to combining text data:

- **The + Operator:** A straightforward syntactical approach where the plus symbol is used as a symbolic shortcut to "add" one string to another.
- **The concat() Method:** A functional approach where a string variable invokes a formal method to append another string.

Syntax for concat(): `String variable_1 = variable_2.concat(variable_3);`

Integrated Programming Example

The following program demonstrates how both methods achieve identical results through different syntactical paths:

```
// String concatenation
class ExStr
{
    public static void main(String arg[])
    {
        String str1, str2, str3;
        str1 = "Computer ";
        str2 = "Science";

        // Method 1: Using the + operator
        str3 = str1 + str2;
        System.out.println(str3);

        // Method 2: Using the concat() method
        str1 = "Computer ";
        str2 = "Science";
        str3 = str1.concat(str2);
        System.out.println(str3);
    }
}
```

Output Analysis: Both operations yield the result **"Computer Science"**. Despite the differing syntax, the underlying result is the allocation of a new string representing the union of the inputs.

ABHILASH B J

Java Expressions

Combination of operators, constants and variables are called expressions.

Classification of Expressions

- **Arithmetic expressions:** Focus on mathematical calculations and numerical results.
- **Relational expressions:** Focus on comparing data points to establish equality or order.
- **Logical expressions:** Focus on evaluating complex truth conditions and boolean logic.

Arithmetic Expressions

Arithmetic expressions produce numeric values (such as `integer`, `float`, and `double`). They operate across three primary modes:

1. **Integer Arithmetic:**
   - **Behavior**: Occurs when all operands are integers. Operations yield an integer result. In integer division, any fractional part of the quotient is truncated.
   - **Examples**:
     - `5 / 3` yields `1`
     - `4 / 7` yields `0`
     - `5 / 3 + 4 / 7` yields `1`
2. **Real Arithmetic:**
   - **Behavior**: Occurs when all operands are real numbers (floating-point). Operations yield a floating-point value.
   - **Examples**:
     - `3.0 / 5.0` yields `0.6`
     - `3.0 / 5.0 * 4.0` yields `2.4`
3. **Mixed Arithmetic:**
   - **Behavior**: Occurs when operands are of different data types. Java converts operands to the same data type before performing the operation.
   - **Examples**:
     - `4 / 5 + 3.2` yields `3.2`
     - `5.5 + 5` yields `10.5`

Relational Expressions

- **Definition**: Used to compare the values of two arithmetic expressions.
- **Output**: Produces a logical result of type boolean (`true` or `false`).
- **Example**:
  - `c = a > b;` (where `a` and `b` are `integer`, `float`, or `double` operands; `c` receives `true` or `false`).

ABHILASH B J

Logical Expressions

- **Definition**: Used to express logical computations.
- **Output**: Produces a logical result (`true(1)` or `false(0)`).
- **Example**:
  - `c = a && b;` (where `a` and `b` are `integer`, `float`, or `double` operands; `c` receives `true` or `false`).

Difference Between Expressions and Statements

Concept	Description	Syntax / Form	Examples

Expression	Contains variables, constants, and mathematical operators combined to produce a meaningful result.	Operators and operands without mandatory assignment.	`x + y`
			`a = 10`
			`a + b * c`

Statement	Formed when an expression is assigned to a variable.	`variable = expression;`	`z = x + y;` `c = a + 10;`

Evaluation of Expressions, Precedence, and Associativity

Mechanics of Expression Evaluation

- Expressions are evaluated using assignment statements of the form: `variable = expression;`
- When an assignment statement is encountered, the expression on the right-hand side is evaluated first, and the result replaces the previous value of the left-hand variable.
- All variables in an expression must be assigned values prior to evaluation.
- Blank spaces around operators are optional and serve only to enhance code readability.

Program Demonstration: Basic Expression Evaluation

```
// Expression evaluation
class Evaluation
{
    public static void main(String arg[])
    {
        int a, b, c, x = 10, y = 15, z = 5, w = 5;
        a = x * y - z;
```

ABHILASH P J

```
        b = y / z * x;
        c = x - y / z + w;
        System.out.println("a=" + a);
        System.out.println("b=" + b);
        System.out.println("c=" + c);
    }
}
```

Program Output:

- `a = 145` (Calculated as `10 * 15 - 5 = 150 - 5`)
- `b = 30` (Calculated as `15 / 5 * 10 = 3 * 10`)
- `c = 12` (Calculated as `10 - (15 / 5) + 5 = 10 - 3 + 5`)

Operator Precedence and Associativity Rules

Java uses explicit rules to determine the order in which operators in an expression are evaluated:

1. **Precedence**: When two operators share an operand, the operator with the higher precedence level is evaluated first (e.g., `1 + 2 * 3` is treated as `1 + (2 * 3)`).
2. **Associativity**: When two operators in an expression have the same precedence level, evaluation proceeds according to their associativity rule:
    - **Right-to-Left**: E.g., `x = y = z = 17` is evaluated as `x = (y = (z = 17))`.
    - **Left-to-Right**: E.g., `72 / 2 / 3` is evaluated as `(72 / 2) / 3`.
3. **Parentheses**: Expressions within explicit parentheses hold the highest priority. Inner nested parenthesized sub-expressions evaluate first.

Complete Operator Precedence and Associativity Hierarchy (or) Operator Priority.

Level	Operator	Description	Associativity

1	`[]`
	`.`
	`()`
	`++`
	`--`
		Access array element.
		Access object member.
		Invoke a method.
		Post-increment.
		Post-decrement.
			Left to right

2	`++`
	`–`
	`+`
	`-`
	`!`
		Pre-increment
		Pre-decrement
		Unary plus
		Unary minus
		Logical NOT
		Bitwise NOT
			Right to left

ABHILASH B J

	`~`		
3	`()` `new`	Cast Object creation	Right to left

4	`*`
	`/`
	`%`
		Multiplicative	Left to right

5	`+`
	`-`
	`+`
		Additive
		Additive
		String concatenation
			Left to right

6	`<<`
	`>>`
	`>>>`
		Shift	Left to right

7	`<`
	`<=`
	`>`
	`>=`
	`instanceof`
		Relational
		Type comparison
			Left to right

8	`==` `!=`	Equality	Left to right
9	`&`	Bitwise AND	Left to right
10	`^`	Bitwise XOR	Left to right
11	`|`	Bitwise OR	Left to right
12	`&&`	Conditional AND	Left to right
13	`||`	Conditional OR	Left to right
14	`?:`	Conditional	Right to left

15	`=`
	`+=`
	`-=`
	`*=`
	`/=`
	`%=`
	`&=`
		Assignment	Right to left

ABHILASH B J

	^= |= <<= >>= >>>=		

Step-by-Step Expression Evaluation Example

Given variables: `int r = 5, x = 10, a = 2, b = 3, y;`
Expression: `y = r + x + a * b`

Evaluation Steps:

1. `a * b` is calculated first due to multiplicative precedence (`2 * 3 = 6`).
2. `r + x` is calculated from left to right (`5 + 10 = 15`).
3. The remaining addition is calculated and assigned to `y` (`15 + 6 = 21`).

Code Implementation (`class Expre`)

```
// Usage of precedence Operator
class Expre
{
    public static void main(String arg[])
    {
        int r = 5, x = 10, a = 2, b = 3, y;
        y = r + x + a * b;
        System.out.println("Y = " + y);
    }
}
```

Output:

```
Y = 21
```

ABHILASH B J

Mathematical Functions (`Math` Class)

Java provides mathematical functions within the `java.Math.lang` package. All functions in the `Math` class are **static** and are called directly using the class name without requiring object instantiation.

Overview of Mathematical Functions

Function	Description
`sin(x)`	Sine of `x` (`x` in radians)
`cos(x)`	Cosine of `x` (`x` in radians)
`tan(x)`	Tangent of `x` (`x` in radians)
`asin(x)`	Inverse sine of `x`
`acos(x)`	Inverse cosine of `x`
`atan(x)`	Inverse tangent of `x`
`sinh(x)`	Hyperbolic sine of `x`
`cosh(x)`	Hyperbolic cosine of `x`
`tanh(x)`	Hyperbolic tangent of `x`
`exp(x)`	e raised to the power of `x`
`log(x)`	Natural logarithm of `x`
`log10(x)`	Base 10 logarithm of `x`
`pow(x, y)`	`x` raised to the power of `y`
`sqrt(x)`	Square root of `x`
`abs(x)`	Absolute value of `x`
`ceil(x)`	Rounds `x` up to the nearest integer
`floor(x)`	Rounds `x` down to the nearest integer

ABHILASH B J

Detailed Function Specifications & Examples

- **`pow()`**
  - **Description**: Calculates a number raised to a power. Takes two arguments: `argument-1` (base) and `argument-2` (power).
  - **Syntax**: `pow(argument-1, argument-2);`
  - **Examples**:
    - `pow(5, 2)` returns `25.0`
    - `pow(3, 3)` returns `27.0`

- **`sqrt()`**
  - **Description**: Calculates the square root of a given argument.
  - **Syntax**: `sqrt(argument);`
  - **Examples**:
    - `sqrt(25)` returns `5.0`
    - `sqrt(36)` returns `6.0`

- **`sin()`**
  - **Description**: Calculates the sine value of a given argument.
  - **Syntax**: `sin(argument);`
  - **Examples**:
    - `sin(90)` returns `1.0`
    - `sin(30)` returns `0.5`

- **`cos()`**
  - **Description**: Calculates the cosine value of a given argument.
  - **Syntax**: `cos(argument);`
  - **Examples**:
    - `cos(0)` returns `1.0`
    - `cos(60)` returns `0.5`

- **`ceil()`**
  - **Description**: Calculates the nearest integer greater than the passed argument (rounds up).
  - **Syntax**: `ceil(argument);`
  - **Examples**:
    - `ceil(5.3)` returns `6.0`
    - `ceil(75.3)` returns `76.0`

- **`floor()`**
  - **Description**: Calculates the nearest integer less than the passed argument (rounds down).
  - **Syntax**: `floor(argument);`
  - **Examples**:

ABHILASH B J

        - `floor(5.3)` returns `5.0`
        - `floor(75.3)` returns `75.0`

- **`fabs()`**
    - **Description**: Computes the absolute value of a floating-point argument.
    - **Syntax**: `fabs(argument);`
    - **Examples**:
        - `fabs(-10)` returns `10.0`
        - `fabs(10)` returns `10.0`

Mathematical Functions Program & Exact Output Demonstration

Code Implementation (`class ExMath`)

```
// Usage of mathematical function
class ExMath
{
    public static void main(String arg[])
    {
        int x = 5;
        System.out.println("Sin of x = " + Math.sin(x));
        System.out.println("Cosine of x = " + Math.cos(x));
        System.out.println("Tangent of x = " + Math.tan(x));
        System.out.println("Inverse Sin of x = " + Math.asin(x));
        System.out.println("Inversse cosine of x = " + Math.acos(x));
        System.out.println("Inverse tangent of x = " + Math.atan(x));
        System.out.println("Hyperbolic Sin of x = " + Math.sinh(x));
        System.out.println("Hyperbolic Cosine of x = " + Math.cosh(x));
        System.out.println("Hyperbole tangent of x = " + Math.tanh(x));
        System.out.println("Exponential(e) power of x = " + Math.exp(x));
        System.out.println("Natural logarithm of x = " + Math.log(x));
        System.out.println("Base 10 logarithm of x = " + Math.log10(x));
        System.out.println("X to the power of 2 = " + Math.pow(x, 2));
        System.out.println("Square root of x = " + Math.sqrt(x));
        System.out.println("Absolute value of x = " + Math.abs(x));
        System.out.println("Ceil value of x = " + Math.ceil(x));
        System.out.println("Floor value of x = " + Math.floor(x));
    }
}
```

ABHILASH B J

Executed Output (When x = 5)

```
Sin of x = -0.9589242746631385
Cosine of x = 0.28366218546322625
Tangent of x = -3.380515006246586
Inverse Sin of x = NaN
Inversse cosine of x = NaN
Inverse tangent of x = 1.373400766945016
Hyperbolic Sin of x = 74.20321057778875
Hyperbolic Cosine of x = 74.20994852478785
Hyperbole tangent of x = 0.9999092042625951
Exponential(e) power of x = 148.4131591025766
Natural logarithm of x = 1.6094379124341003
Base 10 logarithm of x = 0.6989700043360189
X to the power of 2 = 25.0
Square root of x = 2.23606797749979
Absolute value of x = 5
Ceil value of x = 5.0
Floor value of x = 5.0
```

ABHILASH B J

THANK YOU

ABHILASH B J